

Cognitive Elastic Vertical Memory Management for Kubernetes-based Stateful Applications

Science and Technology Advancement – Assessment & Review (STAAR)

Ritesh Kumar Singh
(Regd. No. 2301020845)

under the supervision of
Dr. Rakesh Kumar Ranjan



Department of Computer Science and Engineering
C.V. Raman Global University, Bhubaneswar
Odisha, 752054, India

March 2026

Abstract

Stateful cloud-native services such as databases, queues, and cache-intensive middle tiers remain difficult to autoscale safely because memory is a hard resource in Kubernetes: under-allocation creates Out-of-Memory (OOM) failures, whereas over-allocation inflates cluster cost and reduces packing efficiency. Standard Vertical Pod Autoscaler (VPA) improves long-horizon right-sizing, but it is primarily recommendation oriented, reactive, and historically tied to disruptive actuation semantics. The Elastic Vertical Memory Management (EVMM) approach of Kang and Yu introduced a proactive memory-centric control loop for Kubernetes-based stateful applications, yet its behavior still depends on uniform trend logic and fixed thresholds that react slowly to abrupt phase changes. This report presents **Cognitive Elastic Vertical Memory Management (Cognitive-EVMM)**, an improved control framework that augments EVMM with workload-state classification, hybrid forecasting, adaptive thresholds, confidence-aware fallback, and priority-based memory allocation under node pressure.

The proposed system monitors high-frequency memory telemetry from Prometheus, models short-term demand using a hybrid Exponential Moving Average and linear-trend predictor, classifies each control interval as *steady*, *trending*, or *volatile*, and then selects a state-specific scaling policy. A cognitive decision engine combines usage ratio, rate of change, forecast confidence, and memory priority to produce an actuation score for in-place vertical resizing. The design explicitly targets restart avoidance for stateful services and introduces an OOM-guard panic path for near-limit or high-velocity memory bursts. Experimental analysis using the project implementation, synthetic and injected spikes, and comparative plots shows that Cognitive-EVMM reduces spike response latency from 240 s for standard VPA and 72 s for base EVMM to 12 s, improves resource-efficiency score from 62% to 84% over the EVMM baseline, and achieves approximately 35% lower memory waste than the base paper under mixed workloads. The controller also shifts OOM safety margins away from the danger zone while preserving tighter average slack than conventional VPA, yielding a stronger cost-performance-reliability balance for Kubernetes-based stateful applications.

Keywords: Kubernetes; Vertical Pod Autoscaler; EVMM; Stateful Workloads; In-place Pod Resize; Predictive Scaling; Cognitive Control; Memory Elasticity.

Contents

1	Introduction	1
1.1	Background: Memory Elasticity in Cloud-Native Platforms	1
1.2	Horizontal Scaling versus Vertical Scaling	1
1.3	Limitations of Standard VPA	1
1.4	Motivation for Cognitive EVMM	2
1.5	Research Problem and Objectives	2
1.6	Key Contributions	3
2	Related Work	4
2.1	Kubernetes Vertical Pod Autoscaler	4
2.2	ElasticDocker	4
2.3	RUBAS and Non-Disruptive Vertical Autoscaling	4
2.4	HyScale and Hybrid Autoscaling	4
2.5	EVMM Baseline by Kang and Yu	4
2.6	Research Gaps	5
3	System Architecture	7
3.1	Architectural Overview	7
3.2	Monitoring Module	7
3.3	Prediction Engine	7
3.4	Cognitive Decision Engine	8
3.5	Scaling Controller	8
3.6	End-to-End Workflow	8
4	Proposed Methodology	9
4.1	Problem Formulation	9
4.2	Priority Scoring Model	10
4.3	Predictive Scaling Mechanism	10
4.4	Adaptive Threshold Logic	11
4.5	Cognitive Workload Classification	11
4.6	Cognitive Decision Function	11
4.7	OOM-Guard and Panic Scaling	12
4.8	Algorithmic Summary	12
4.9	Why the Method Improves on VPA and EVMM	13
5	Experimental Setup	14
5.1	Cluster Environment	14
5.2	Target Workloads	14
5.3	Compared Baselines	15

5.4	Evaluation Metrics	15
5.4.1	Memory Scaling Efficiency	15
5.4.2	Resource Waste	15
5.4.3	OOM Safety Margin	16
5.4.4	Scaling Latency	16
5.4.5	Cost Savings Rate	16
5.4.6	Final Evaluation Score	16
5.5	Threat Model and Assumptions	16
6	Results and Analysis	17
6.1	Overall Comparative Summary	17
6.2	Memory Scaling Comparison	17
6.3	Fine-Grained Memory Trace Interpretation	18
6.4	Memory Waste and Resource Slack	18
6.5	OOM Margin Distribution and Safety	20
6.6	Response Latency Comparison	21
6.7	Cost Savings Rate	21
6.8	Evaluation Summary Metrics	22
6.9	Final Scalability and Reliability Score	23
6.10	Interpretive Synthesis	23
7	Discussion	27
7.1	Strengths of the Proposed System	27
7.2	Trade-offs and Operational Considerations	27
7.3	Real-World Applicability	27
7.4	Validity Considerations	28
7.5	Implications for Kubernetes Resource Management	28
8	Conclusion	28
	Acknowledgments	30
	References	31

List of Figures

1	Memory scaling comparison among standard VPA, base EVMM, and proposed Cognitive-EVMM.	19
2	Fine-grained view of workload demand and dynamic memory limits during mixed workload phases.	19
3	Aggregate memory waste comparison across evaluated methods.	20
4	Accumulated resource-waste behavior over time.	21
5	OOM safety margin distribution across compared methods.	22
6	Latency behavior under dynamic scaling actions.	23
7	Response latency to a sudden 900 MiB spike.	24
8	Comparative cost savings rate derived from reduced memory-hours.	25
9	Evaluation summary metrics across VPA, base EVMM, and Cognitive-EVMM.	25
10	Final scalability and reliability score.	26

List of Tables

1	Problem-to-solution mapping in Cognitive-EVMM.	3
2	Comparison of related approaches and the proposed system.	6
3	System modules and data flow.	9
4	Controller parameters used in the project prototype.	14
5	Experimental environment and workload settings.	17
6	Headline comparative results from the experimental evaluation. Approximate slack values are read from the generated comparison plots and summary artifacts.	18

1. Introduction

1.1. Background: Memory Elasticity in Cloud-Native Platforms

Kubernetes has become the dominant orchestration substrate for microservices because it unifies deployment, service discovery, fault recovery, and scale control in a declarative control plane. Most production clusters already rely on Horizontal Pod Autoscaler (HPA) for throughput scaling and cluster-level autoscalers for node elasticity. However, memory-heavy stateful services do not benefit from horizontal scaling as directly as stateless request handlers. Replicating a front-end worker can absorb traffic; replicating a stateful database replica or stream-processing shard often requires data movement, consistency coordination, cache rewarming, and storage-aware placement. Consequently, many real-world workloads remain bottlenecked by *vertical* resource adequacy rather than pod-count elasticity.

For memory in particular, the operational stakes are high. CPU overuse usually leads to throttling; memory overuse leads to kernel reclamation, cgroup pressure, latency amplification, and eventually OOM termination. These effects are especially harmful for stateful workloads whose working sets are expensive to reconstruct. A vertical memory controller must therefore satisfy two objectives that are naturally in tension: it must maintain sufficient headroom to protect the working set, yet it must avoid persistent slack that wastes node memory and reduces bin-packing density.

1.2. Horizontal Scaling versus Vertical Scaling

The distinction between horizontal and vertical autoscaling is central to the motivation of this work.

- **Horizontal scaling** increases or decreases the number of pod replicas. It is effective for embarrassingly parallel request processing, but it is less suitable for monolithic state shards, primary databases, in-memory caches, and single-threaded state coordinators.
- **Vertical scaling** changes per-pod CPU and memory allocations. It is the natural strategy when the application cannot be split quickly, when state transfer is expensive, or when a single pod must preserve session affinity and warm memory state.

In practice, stateful services often require both. Yet when memory demand rises abruptly, vertical scaling is usually the first line of defense because it can increase the survivability of the running process without re-partitioning the application. The challenge is that vertical scaling has historically been slower and more disruptive than horizontal scaling in Kubernetes, especially when resource updates require pod eviction or recreation.

1.3. Limitations of Standard VPA

The upstream Kubernetes Vertical Pod Autoscaler improves rightsizing by recommending or applying CPU and memory request changes based on observed historical usage [2]. Its design is valuable for long-horizon efficiency improvement, but several limitations remain critical for stateful memory management:

- 1) **Reactive behavior.** VPA is oriented around accumulated observation windows and recommender outputs, not short-horizon burst anticipation.
- 2) **Weak workload awareness.** The same general policy is applied regardless of whether the workload is stable, trending, or highly volatile.
- 3) **Operational disruption.** Although modern Kubernetes now supports in-place resize capabilities [3, 4], restart-free behavior is not universally guaranteed for every resize case, and conventional VPA workflows historically depended on pod eviction or recreation.
- 4) **No memory-priority arbitration.** VPA does not reason explicitly about which pods should receive scarce memory first under node pressure.

These characteristics make standard VPA useful for baseline rightsizing, but insufficient for aggressive stateful memory elasticity where burst survivability and sub-minute response are required.

1.4. Motivation for Cognitive EVMM

The EVMM baseline by Kang and Yu addressed an important gap by treating vertical memory scaling as a proactive control problem for stateful Kubernetes applications [1]. It introduced elastic scaling logic that follows observed demand more closely than static VPA policies and explicitly targets stateful behavior. However, the baseline still exhibits two structural limitations in bursty environments. First, a single predictive regime is applied to all workload phases, even though steady nightly traffic, monotonic ramp-up, and instantaneous spikes require different controller gains. Second, fixed decision thresholds create either lag or oscillation when cluster conditions and workload volatility change simultaneously.

The core motivation of Cognitive-EVMM is therefore to make the control loop *context aware*. Instead of asking only “How much memory was recently consumed?”, the proposed system also asks:

- Is the workload currently steady, trending, or volatile?
- How confident is the short-term forecast?
- How much node pressure exists, and which pod should receive memory first?
- Should the controller optimize primarily for efficiency, anticipation, or safety in the current interval?

1.5. Research Problem and Objectives

This report addresses the following research problem:

How can Kubernetes-based stateful applications be vertically memory-scaled in a proactive, low-waste, restart-avoiding, and OOM-safe manner under mixed workload dynamics?

To answer this problem, the report pursues four objectives:

- 1) design a cognitive extension of EVMM that detects workload phase changes in real time;

Table 1: Problem-to-solution mapping in Cognitive-EVMM.

Challenge	Observed limitation	Cognitive-EVMM response
Burst memory growth	Trend-only prediction lags abrupt spikes	Rate-of-change aware classification and panic path
Long-lived slack	Conservative overprovisioning wastes memory	Steady-state policy favors tight usage-centric limits
Uniform control law	Same scaling weights for all workload phases	State-specific weighting for steady, trending, and volatile modes
Low-confidence forecast	Prediction error can amplify wrong actions	Confidence-aware fallback boosts direct usage influence
Node contention	No explicit arbitration of scarce memory	Priority score ranks pods by urgency and criticality
Disruptive resizing	Stateful pods cannot tolerate frequent restarts	In-place resize oriented actuation with conservative downscale semantics

- 2) formulate a mathematically grounded scaling policy with adaptive thresholds and memory-priority awareness;
- 3) implement the control loop in a Kubernetes and Prometheus based prototype;
- 4) evaluate the design against standard VPA and the base EVMM approach using latency, waste, safety, and reliability metrics.

1.6. Key Contributions

The main contributions of this report are summarized below.

- **Cognitive control extension.** A state-aware controller is proposed to distinguish steady, trending, and volatile workload phases and to change scaling weights accordingly.
- **Hybrid prediction model.** The work combines exponential smoothing with short-term trend inference so that the controller remains stable during smooth periods and still reacts early during ramp-up.
- **Adaptive thresholding.** Scale-up and scale-down thresholds are adjusted online using variance, forecast confidence, and node pressure rather than remaining fixed.
- **Priority-based memory allocation.** A normalized priority score is introduced to arbitrate memory allocation under contention using memory intensity, CPU support, usage variance, and node pressure.
- **Restart-avoidance oriented actuation.** The controller is designed around in-place vertical resizing semantics and therefore targets continuity for stateful services whenever the platform allows it.
- **Comprehensive comparative evaluation.** The report provides graph-by-graph analysis of memory scaling fidelity, resource slack, OOM margin distribution, latency, cost savings, and a final scalability-reliability score.

2. Related Work

2.1. Kubernetes Vertical Pod Autoscaler

The Kubernetes autoscaling ecosystem includes HPA, Cluster Autoscaler, and VPA. VPA focuses on right-sizing pod CPU and memory requests over time and is implemented through recommender, updater, and admission components in the Kubernetes autoscaler project [2]. Its principal strength lies in improving chronic over- or under-requesting. However, VPA does not fundamentally solve the control-theoretic problem of short-horizon memory surges in stateful services. From a research perspective, VPA is a valuable systems baseline rather than a complete solution for rapid vertical memory elasticity.

2.2. ElasticDocker

ElasticDocker pioneered autonomic vertical elasticity for containerized applications by adjusting CPU and memory allocations online based on threshold crossings [5]. Its contributions were important because they demonstrated that containers can be vertically adjusted with less disruption than virtual machine era scaling methods. Nonetheless, ElasticDocker predates contemporary Kubernetes-native in-place resize capabilities, does not focus specifically on stateful microservices in Kubernetes, and relies primarily on threshold-triggered behavior rather than adaptive cognitive control.

2.3. RUBAS and Non-Disruptive Vertical Autoscaling

Rattihalli *et al.* investigated non-disruptive vertical autoscaling and resource estimation in Kubernetes [6]. Their work is valuable because it highlighted the feasibility of modifying resource allocations without always recreating the pod and quantified estimation challenges in real clusters. In that sense, it forms an important bridge between static rightsizing and truly dynamic vertical control. Yet its emphasis is on feasibility and estimation rather than on memory-specific cognitive decision-making, state classification, or cost-reliability tradeoff optimization.

2.4. HyScale and Hybrid Autoscaling

HyScale combined horizontal scaling, vertical scaling, and network-level reasoning for Dockerized microservices in cloud data centers [7]. The paper demonstrated that purely horizontal or purely vertical scaling is often suboptimal when service chains and network bottlenecks are involved. This insight remains relevant here: stateful systems also require coordinated reasoning across multiple resource dimensions. However, HyScale primarily targets service-level scaling orchestration rather than fine-grained memory control for a single stateful pod’s working set.

2.5. EVMM Baseline by Kang and Yu

The closest antecedent to this work is EVMM, which targets elastic vertical memory management for container-based stateful applications in Kubernetes [1]. EVMM recognizes

that memory is both performance critical and failure sensitive in stateful services. Its baseline contribution is to move beyond static VPA-style resizing and to use shorter telemetry intervals together with proactive logic for memory adjustment. The base implementation in this project captures that philosophy through a memory-priority score, trend-aware scale-up logic, and reclaim policy under low utilization.

Despite its relevance, the baseline reveals several limitations under mixed dynamics:

- the same general scaling logic is reused across smooth ramps and abrupt spikes;
- decision thresholds are largely fixed;
- prediction confidence is not explicitly used to attenuate risky forecasts;
- emergency handling remains less sensitive to sudden rate-of-change surges than desired for strict OOM protection.

2.6. Research Gaps

The literature synthesis reveals four research gaps that motivate Cognitive-EVMM.

- 1) **Phase-unaware decision loops.** Existing methods usually treat recent telemetry as a homogeneous signal and do not separate stable, trending, and volatile workload phases.
- 2) **Insufficient memory-specific urgency modeling.** Many systems optimize generic resource usage but do not explicitly prioritize memory survivability for stateful pods.
- 3) **Weak coupling between prediction and confidence.** Forecasts are often used directly even when their statistical reliability is poor.
- 4) **Limited contention-aware allocation.** Cluster-level pressure is often ignored during per-pod vertical decision-making.

Table 2: Comparison of related approaches and the proposed system.

Approach	Platform focus	Primary scaling logic	Workload awareness	Restart avoidance	Key limitation for stateful memory control
Standard VPA [2]	Kubernetes	Historical recommendation and update loop	None	Limited and historically eviction oriented	Reactive and not tuned for burst memory survivability
ElasticDocker [5]	Docker	Threshold-based vertical elasticity	Limited	Partial	Not Kubernetes-native and not memory-cognitive
RUBAS / non-disruptive vertical autoscaling [6]	Kubernetes	Estimation and non-disruptive resizing feasibility	Limited	Yes, when supported	Focused on feasibility rather than adaptive decision logic
HyScale [7]	Dockerized microservices	Hybrid horizontal, vertical, and network scaling	Service-level	Indirect	Not specialized for memory safety of stateful pods
EVMM [1]	Kubernetes stateful apps	Predictive elastic memory control	Partial	Better than VPA	Fixed thresholds and limited spike cognition
Cognitive-EVMM	Kubernetes stateful apps	Hybrid prediction, adaptive thresholds, cognitive decision engine	Steady / trending / volatile	Designed for in-place actuation	Requires accurate telemetry and careful downscale policy

3. System Architecture

3.1. Architectural Overview

Cognitive-EVMM is organized as a closed-loop control pipeline deployed alongside Kubernetes workloads. The architecture follows the monitor-analyze-decide-act pattern but introduces a cognitive layer between prediction and actuation. The full pipeline consists of four core modules:

- 1) **Monitoring module**
- 2) **Prediction engine**
- 3) **Cognitive decision engine**
- 4) **Scaling controller**

At a high level, Prometheus collects memory telemetry from the target pod, the monitoring module maintains a bounded history window, the prediction engine estimates near-future demand, the cognitive engine classifies workload state and computes a scaling score, and the controller applies an in-place resource patch subject to safety constraints and cooldown rules.

3.2. Monitoring Module

The implementation queries `container_memory_working_set_bytes` from Prometheus every 10s, converts the metric to MiB, and appends it to a rolling history. The history length in the implementation is capped at 60 samples, which corresponds to approximately 10 minutes of recent context. In addition to raw memory usage, the controller derives:

- utilization ratio $u_i(t) = U_i(t)/L_i(t)$,
- instantaneous rate of change $r_i(t)$,
- short-window variance and volatility indicators,
- current memory limit from the Kubernetes deployment spec.

The monitoring module therefore serves two roles: it produces operational telemetry for scaling decisions, and it exposes research metrics such as current score, workload class, forecast confidence, savings in MB-hours, and panic count via Prometheus-compatible gauges.

3.3. Prediction Engine

The prediction engine estimates near-future memory demand over a short horizon H . The implementation uses a hybrid approach:

- 1) an Exponential Moving Average (EMA) captures short-term smoothed behavior and filters high-frequency noise;
- 2) a linear regression model over the recent history estimates the trend slope and R^2 confidence;
- 3) both signals are combined to form a hybrid demand forecast.

This design is intentionally lightweight. It avoids the runtime overhead and training requirements of heavy machine learning models while still encoding two important intu-

itions: recent usage should matter more than older usage, and persistent ramps should influence proactive limit changes before threshold breach occurs.

3.4. Cognitive Decision Engine

The cognitive engine is the central innovation of the proposed architecture. It first classifies the workload into one of three behavioral states:

- **Steady**: low variance, weak directional trend, low rate fluctuation.
- **Trending**: strong linear direction with sufficiently high forecast confidence.
- **Volatile**: bursty or rapidly changing usage with high rate variance or danger-zone urgency.

After classification, the engine selects a weighting vector for decision features. In steady mode, direct utilization dominates because efficiency is the primary objective. In trending mode, forecasted demand receives larger weight to anticipate continued ramp-up. In volatile mode, rate-of-change and safety take precedence so that the controller reacts before OOM conditions develop.

The implementation also includes confidence-aware fallback: when forecast confidence falls, trend weight is attenuated and direct usage receives more weight. This prevents unstable actuation when linear fit quality deteriorates during abrupt behavioral transitions.

3.5. Scaling Controller

The scaling controller is the actuation layer that patches Kubernetes resource limits and requests. In the prototype, the controller updates the target deployment through the Kubernetes API and respects four guardrails:

- 1) **minimum and maximum memory bounds** (128 MiB to 4096 MiB);
- 2) **cooldown control** (90 s) to limit oscillation;
- 3) **panic scaling** when near-limit danger is detected;
- 4) **conservative scale-down** to avoid repeated shrinking and re-expanding.

The sample workload manifest explicitly enables memory `resizePolicy` with `restartPolicy: NotRequired`, making the configuration compatible with restart-avoiding vertical adjustments where supported by the cluster version and runtime implementation. In production terms, this is critical: upward resizing is prioritized because it directly protects the process working set, while downscaling is intentionally more conservative because some memory decrease cases may still be operationally constrained even with modern in-place resize support.

3.6. End-to-End Workflow

The end-to-end workflow for one control interval is summarized below.

- 1) Prometheus samples the target pod every 10 s.
- 2) The monitoring module fetches memory usage and updates the history window.
- 3) The predictor computes EMA, trend slope, and forecast confidence.
- 4) The classifier labels the interval as steady, trending, or volatile.

Table 3: System modules and data flow.

Module	Inputs	Outputs	Function in control loop
Monitoring module	Prometheus memory samples, current pod limit, recent history	Usage ratio, ROC, variance, telemetry gauges	Converts raw observability data into controller features
Prediction engine	Time-series history	Hybrid forecast, slope, confidence	Estimates short-horizon demand while filtering noise
Cognitive decision engine	Usage, ROC, prediction, confidence, node pressure, priority	State label, decision score, panic flag	Chooses control regime and balances efficiency against safety
Scaling controller	Decision score, bounds, cooldown state, pod spec	New requests and limits patch	Applies in-place vertical resizing with guardrails

- 5) The decision engine computes a cognitive score and evaluates OOM-guard conditions.
- 6) The scaling controller computes the new limit and patches Kubernetes resources if cooldown and bound checks pass.
- 7) Exported metrics record the actuation outcome for visualization and research analysis.

4. Proposed Methodology

4.1. Problem Formulation

Consider a node or cluster segment hosting N stateful pods. Let $U_i(t)$ denote the observed memory usage of pod i at control instant t , and let $L_i(t)$ denote its configured memory limit. The control objective is to choose $L_i(t + \Delta)$ such that memory waste is minimized while OOM risk, scaling latency, and disruptive actuation are constrained.

We formulate the optimization target as

$$\min_{\{L_i\}} J(t) = \lambda_1 \sum_{i=1}^N [L_i(t) - U_i(t)]^+ + \lambda_2 \sum_{i=1}^N \mathcal{R}_i^{\text{oom}}(t) + \lambda_3 \sum_{i=1}^N \mathcal{T}_i^{\text{scale}}(t) + \lambda_4 \sum_{i=1}^N \mathcal{C}_i^{\text{restart}}(t), \quad (1)$$

subject to

$$\sum_{i=1}^N L_i(t) \leq M_{\text{node}}^{\text{avail}}(t), \quad (2)$$

$$L_i^{\min} \leq L_i(t) \leq L_i^{\max}. \quad (3)$$

Here, $[x]^+ = \max(x, 0)$ measures positive slack, $\mathcal{R}_i^{\text{oom}}(t)$ is a risk proxy linked to low safety margin, $\mathcal{T}_i^{\text{scale}}(t)$ models actuation delay, and $\mathcal{C}_i^{\text{restart}}(t)$ captures the operational

penalty of restart-inducing resizes. The coefficients $\lambda_1, \dots, \lambda_4$ encode system preference. For stateful services, λ_2 and λ_4 are typically higher than in stateless deployments.

4.2. Priority Scoring Model

To arbitrate memory allocation under shared node pressure, Cognitive-EVMM assigns each pod a normalized priority score:

$$P_i(t) = \alpha \hat{m}_i(t) + \beta \hat{c}_i(t) + \gamma \hat{\sigma}_i(t) + \delta \hat{\rho}_n(t) + \eta \omega_i, \quad (4)$$

where

- $\hat{m}_i(t)$ is normalized memory pressure, typically $U_i(t)/L_i(t)$;
- $\hat{c}_i(t)$ is normalized CPU usage or support demand;
- $\hat{\sigma}_i(t)$ is normalized short-window memory variance;
- $\hat{\rho}_n(t)$ is node pressure, such as used memory over allocatable node memory;
- ω_i is a service criticality factor for business or statefulness priority.

The weights satisfy $\alpha + \beta + \gamma + \delta + \eta = 1$. In the single-deployment prototype, ω_i can be treated as constant, but the formulation generalizes naturally to multi-tenant clusters. Compared with base EVMM, the proposed score explicitly links pod-level urgency to node-level pressure, which becomes important when multiple stateful pods compete for limited memory.

4.3. Predictive Scaling Mechanism

The forecast for memory demand over horizon H is defined as a convex combination of a smoothed signal and a trend signal:

$$\text{EMA}_i(t) = \lambda U_i(t) + (1 - \lambda) \text{EMA}_i(t - \Delta), \quad (5)$$

$$\text{Trend}_i(t + H) = U_i(t) + H \cdot \dot{U}_i(t), \quad (6)$$

$$\hat{U}_i(t + H) = \kappa \text{EMA}_i(t) + (1 - \kappa) \text{Trend}_i(t + H), \quad (7)$$

where $\dot{U}_i(t)$ is the estimated slope from linear regression and $\kappa \in [0, 1]$ balances stability against anticipation. In the prototype, regression confidence is captured through R^2 :

$$\chi_i(t) = \max(0, R_i^2(t)). \quad (8)$$

Confidence then modulates how aggressively the trend component is trusted. If $\chi_i(t)$ is low, the controller falls back toward direct usage and EMA-dominant behavior. This is preferable to blindly following a low-quality forecast.

4.4. Adaptive Threshold Logic

Fixed thresholds are easy to implement but perform poorly across heterogeneous traffic phases. Cognitive-EVMM therefore defines adaptive scale-up and scale-down thresholds:

$$\theta_i^\uparrow(t) = \theta_0^\uparrow - a_1 \hat{\sigma}_i(t) - a_2 \hat{\rho}_n(t) - a_3 [1 - \chi_i(t)], \quad (9)$$

$$\theta_i^\downarrow(t) = \theta_0^\downarrow - b_1 [1 - \chi_i(t)] + b_2 [1 - \hat{\rho}_n(t)]. \quad (10)$$

Equation (9) lowers the scale-up threshold when variance is high, node pressure is high, or confidence in the current model is low. This means the controller becomes more conservative about safety during uncertain or bursty periods. Equation (10) ensures that downscaling is delayed when forecast confidence is low or when node pressure does not justify aggressive reclamation.

4.5. Cognitive Workload Classification

The classifier uses recent telemetry to map each interval into one of three workload states:

$$\mathcal{S}_i(t) = \begin{cases} \text{steady,} & \hat{\sigma}_i(t) < \tau_\sigma \wedge R_i^2(t) < \tau_r \\ \text{trending,} & R_i^2(t) \geq \tau_r \wedge \dot{U}_i(t) > 0 \\ \text{volatile,} & \text{otherwise or } |r_i(t)| > \tau_{\text{roc}}. \end{cases} \quad (11)$$

This classification is intentionally simple enough for fast control-loop execution while remaining semantically expressive. The logic aligns with the implementation: steady workloads prioritize tight resource tracking, trending workloads prioritize forecasted headroom, and volatile workloads prioritize near-term survivability.

4.6. Cognitive Decision Function

For a workload state $k \in \{\text{steady, trending, volatile}\}$, let the state-specific weight vector be

$$\mathbf{w}^{(k)} = [w_u^{(k)}, w_r^{(k)}, w_f^{(k)}, w_p^{(k)}]. \quad (12)$$

The cognitive decision score is then

$$D_i(t) = w_u^{(k)} u_i(t) + w_r^{(k)} \tilde{r}_i(t) + w_f^{(k)} \chi_i(t) \frac{\hat{U}_i(t + H)}{L_i(t)} + w_p^{(k)} P_i(t), \quad (13)$$

where $\tilde{r}_i(t)$ is the normalized rate of change. In the concrete prototype, the weighting pattern follows:

- steady: usage-dominant;
- trending: forecast-dominant;
- volatile: ROC-dominant.

The output score $D_i(t)$ is mapped to a bounded scale factor:

$$s_i(t) = \begin{cases} 1.5, & D_i(t) > \tau_{85} \\ 1.2, & \tau_{70} < D_i(t) \leq \tau_{85} \\ 0.8, & \tau_{25} \leq D_i(t) < \tau_{40} \\ 0.6, & D_i(t) < \tau_{25} \\ 1.0, & \text{otherwise.} \end{cases} \quad (14)$$

The new target limit becomes

$$L'_i(t) = \text{clip}(s_i(t)L_i(t), L_i^{\min}, L_i^{\max}). \quad (15)$$

4.7. OOM-Guard and Panic Scaling

The controller includes an emergency branch for situations where standard score-based resizing may be too slow. If the pod enters the danger zone, defined by

$$\frac{U_i(t)}{L_i(t)} > 0.92 \quad \text{or} \quad |r_i(t)| > \tau_{\text{panic}}, \quad (16)$$

the controller bypasses the normal scoring path and applies a panic multiplier:

$$L_i^{\text{panic}}(t) = \text{clip}(2.0L_i(t), L_i^{\min}, L_i^{\max}). \quad (17)$$

In the repository implementation, near-limit panic is explicit and ROC strongly influences state selection and score; in the plot-based experimental model, the burst path additionally uses an immediate ROC override to emulate aggressive high-velocity reaction. Both interpretations are consistent with the design goal: preventing OOM rather than merely reacting after breach.

4.8. Algorithmic Summary

```

1  Input:
2    telemetry stream, memory bounds, cooldown, prediction horizon H
3  Output:
4    new memory request/limit for target pod
5
6  while controller is active do
7    U_t <- fetch current memory usage from Prometheus
8    L_t <- fetch current memory limit from Kubernetes
9    History <- append(U_t) and trim to rolling window
10
11   r_t <- rate_of_change(History)
12   sigma_t <- variance(History)
13   rho_t <- node_pressure()
```

```

14     P_t <- priority_score(U_t, CPU_t, sigma_t, rho_t, criticality)
15
16     EMA_t <- exponential_average(History)
17     Trend_t, conf_t <- linear_forecast(History, H)
18     U_hat <- hybrid_forecast(EMA_t, Trend_t)
19
20     State <- classify(sigma_t, r_t, conf_t, Trend_t)
21
22     if danger_zone(U_t, L_t, r_t) then
23         L_new <- clip(2.0 * L_t, L_min, L_max)
24     else
25         Weights <- select_state_weights(State)
26         D_t <- weighted_score(U_t/L_t, normalize(r_t),
27                               conf_t * U_hat/L_t, P_t, Weights)
28         Thresholds <- adapt_thresholds(sigma_t, rho_t, conf_t)
29         Scale <- map_score_to_step(D_t, Thresholds)
30         L_new <- clip(Scale * L_t, L_min, L_max)
31     end if
32
33     if cooldown_elapsed() and L_new != L_t then
34         patch_kubernetes_resources(L_new)
35         export_observability_metrics()
36     end if
37
38     sleep(10 seconds)
39 end while

```

Listing 1: Pseudo-code for the proposed Cognitive-EVMM controller.

4.9. Why the Method Improves on VPA and EVMM

The methodology improves on standard VPA because it is *phase aware*, *short-horizon predictive*, and *memory-priority conscious*. It improves on the base EVMM design because it couples prediction with confidence, couples scale thresholds with current variance and node pressure, and separates controller behavior across workload phases instead of using a single trend-centric regime. Conceptually, Cognitive-EVMM is not just “more aggressive EVMM”; it is a different control strategy in which the controller first decides *what kind of memory dynamics* it is observing and then chooses the appropriate scaling law.

Table 4: Controller parameters used in the project prototype.

Parameter	Value	Interpretation
Sampling interval	10 s	Prometheus polling and control-loop cadence
History size	60 samples	Approximately 10 minutes of rolling context
Prediction horizon	120 s	Short-horizon demand lookahead
Minimum memory	128 MiB	Lower safety bound for pod limit
Maximum memory	4096 MiB	Upper cap for resize actions
Cooldown	90 s	Anti-oscillation guard between actions
Danger-zone threshold	92% of limit	OOM-guard trigger
Scale-up steps	1.2, 1.5, 2.0	Moderate, strong, and panic expansion
Scale-down steps	0.8, 0.6	Controlled reclamation for low-demand phases

5. Experimental Setup

5.1. Cluster Environment

The experimental prototype is implemented as a Kubernetes-deployed Python autoscaler using the official Kubernetes Python client, Prometheus API client, Pandas-based feature processing, and scikit-learn linear regression. The autoscaler is packaged as a deployment with RBAC permissions to read and patch deployments in the cluster. The target workload is a sample stateful-like containerized application deployed with explicit memory requests and limits, and with memory resize policy configured to avoid restarts where supported by the cluster runtime.

The experimental environment includes the following practical components:

- Kubernetes deployment for the autoscaler controller;
- Prometheus for metric collection;
- sample workload deployment with resizable memory limits;
- Grafana dashboards and exported controller metrics for observation;
- workload generators for steady, trending, and burst phases.

5.2. Target Workloads

Two workload-generation paths are used in the project artifacts.

- 1) **Synthetic trace generation.** The plot generator combines sinusoidal baseline demand, random noise, a large sudden spike, and a sustained upward trend. This produces a mixed workload containing all three controller states in a single experiment.
- 2) **Live spike injection.** The spike trigger script allocates a large memory object inside the sample application pod and holds it for a fixed period, creating a sharp burst suitable for latency and OOM-guard evaluation.

The mixed traces are intentionally useful for research because they expose three distinct

controller stress cases:

- **steady periods** test whether the controller can reclaim slack instead of holding conservative limits;
- **trend periods** test whether predictive scaling can widen headroom before threshold breach;
- **burst periods** test whether the safety logic acts quickly enough to avoid OOM.

5.3. Compared Baselines

Three systems are compared throughout the report.

- 1) **Standard VPA.** A reactive baseline with relatively delayed response and historically disruptive actuation semantics.
- 2) **Base EVMM.** The implementation derived from the 2025 EVMM paper, using priority scoring and predictive adjustment without cognitive phase switching.
- 3) **Proposed Cognitive-EVMM.** The improved controller with hybrid prediction, adaptive thresholds, workload classification, and OOM-guard logic.

The comparison is therefore deliberately incremental: VPA provides the common Kubernetes baseline, EVMM provides the relevant state-of-the-art research baseline, and Cognitive-EVMM isolates the gains produced by the proposed extensions.

5.4. Evaluation Metrics

The evaluation uses the following metrics.

5.4.1. Memory Scaling Efficiency

Scaling efficiency measures how closely the allocated limit follows actual memory demand without entering unsafe territory. Let slack be

$$S_i(t) = L_i(t) - U_i(t). \quad (18)$$

Low positive slack is desirable; negative slack indicates a demand overshoot and thus heightened OOM risk.

5.4.2. Resource Waste

Cumulative memory waste over experiment horizon T is

$$W_i = \int_0^T [L_i(t) - U_i(t)]^+ dt. \quad (19)$$

This metric captures the economic cost of conservative sizing. The implementation also exports saved MB-hours relative to a maximum limit.

5.4.3. OOM Safety Margin

The instantaneous OOM margin is

$$M_i(t) = L_i(t) - U_i(t). \quad (20)$$

The distribution of $M_i(t)$ indicates whether the controller spends too much time in the danger zone. A system may be efficient on average while still unsafe if lower-tail margins collapse during bursts.

5.4.4. Scaling Latency

Scaling latency is the elapsed time between a meaningful workload demand transition and the controller establishing sufficient new headroom. In the comparative plots, the reference event is a sudden 900 MiB spike. Lower latency directly improves survivability for stateful services.

5.4.5. Cost Savings Rate

Cost savings are evaluated as a normalized relative reduction in consumed memory-hours:

$$\text{CSR} = \frac{C_{\text{baseline}} - C_{\text{method}}}{C_{\text{baseline}}}. \quad (21)$$

Although the report uses memory-centric workloads rather than cloud-provider billing traces, CSR is still a meaningful proxy because memory reservations are a primary driver of node count and instance sizing.

5.4.6. Final Evaluation Score

To summarize multi-objective behavior, a report-level balanced efficiency index (BEI) is defined as

$$\text{BEI} = 0.30E + 0.25S + 0.25R + 0.20Q, \quad (22)$$

where E is normalized efficiency, S is normalized safety, R is responsiveness, and Q is stability/reliability. This score is not used by the controller at runtime; it is used only for comparative research synthesis.

5.5. Threat Model and Assumptions

The experimental design assumes that Prometheus telemetry reflects application memory pressure with sufficiently low lag, that the Kubernetes cluster supports practical in-place memory resize behavior for the chosen workload pattern, and that the target deployment is a valid proxy for a stateful service whose restart cost is materially higher than that of a stateless worker. The experiments therefore emphasize *controller behavior* and *comparative actuation quality*. They do not attempt to model every possible storage engine or scheduler implementation detail.

Table 5: Experimental environment and workload settings.

Aspect	Setting	Description
Autoscaler runtime	Python 3.9 container	Controller runs inside Kubernetes deployment
Metric source	Prometheus	Polls memory working set every 10 s
Workload pod initial request	128 MiB	Initial baseline memory reservation
Workload pod initial limit	256 MiB	Initial memory ceiling for sample app
Observed metric	Working set memory	Main signal used for scaling decisions
Burst injection	800–900 MiB class spike	Used to evaluate latency and safety
Trend window	Up to 120 s lookahead	Forecast horizon for proactive scaling
Baselines	VPA, EVMM, Cognitive-EVMM	Comparative systems under same workload family

6. Results and Analysis

6.1. Overall Comparative Summary

Table 6 summarizes the headline outcomes drawn from the repository plots and comparative evaluation. The most important pattern is not simply that Cognitive-EVMM is “better” on one metric, but that it improves the *shape of the tradeoff*. Standard VPA is safe largely because it is slow and overprovisioned. Base EVMM is more efficient but remains vulnerable to abrupt dynamics. Cognitive-EVMM occupies a more desirable middle region: high efficiency, low latency, and substantially improved lower-tail safety.

6.2. Memory Scaling Comparison

Figure 1 compares the memory limit traces of standard VPA, base EVMM, and Cognitive-EVMM against workload demand. The differences are architecturally revealing.

Standard VPA shows a pronounced stair-step pattern. Limits move upward slowly, remain high for longer than necessary, and only reclaim memory after substantial delay. This behavior is unsurprising for a reactive system: it is designed to avoid oscillation, not to track short-lived memory transients tightly. The result is a large safety envelope, but one paid for through persistent slack.

Base EVMM tracks the workload more closely. The trace validates the usefulness of short-window predictive logic: compared with VPA, the controller rises earlier and reclaims more memory after peaks. However, during the large spike and the later upward ramp, the base EVMM curve still approaches the demand line too closely. This reflects the limitation of using a mostly uniform predictive law for heterogeneous phases. Trend information

Table 6: Headline comparative results from the experimental evaluation. Approximate slack values are read from the generated comparison plots and summary artifacts.

Metric	Standard VPA	Base EVMM	Cognitive-EVMM
Spike response latency	240 s	72 s	12 s
Resource-efficiency score	48%	62%	84%
Average slack (approx.)	210 MiB	120 MiB	78 MiB
Final scalability-reliability score	45/100	65/100	96/100
Balanced efficiency index	0.57	0.71	0.92
Observed qualitative burst behavior	Late and stair-step	Faster but trend-lagged	Early and state aware

helps during ramps, yet it underreacts when the workload changes more quickly than the fitted trend can extrapolate.

Cognitive-EVMM produces the most desirable shape. During calm periods, it collapses the gap between limit and usage, indicating efficient reclamation. During the ramp and burst windows, it expands quickly and earlier than the base EVMM curve. The result is neither the wide slack of VPA nor the thin lower-tail margin of base EVMM, but a context-dependent envelope that expands and contracts according to workload state.

6.3. Fine-Grained Memory Trace Interpretation

The zoomed trace in Figure 2 illustrates the mechanism of improvement more clearly.

The crucial observation is the *ordering* of reactions. VPA first waits, then climbs; base EVMM predicts, then corrects; Cognitive-EVMM classifies, anticipates, and if necessary triggers a safety-oriented response. This ordering matters because OOM avoidance is fundamentally a race between application growth and control-loop delay. By transforming the controller from a single-law predictor into a state-aware policy selector, the proposed system shortens the decision path for volatile intervals.

From a control standpoint, this figure also indicates lower overshoot after burst recovery. Once the workload returns from a spike, the proposed controller does not remain excessively inflated for long. This suggests that the steady-state policy and scale-down logic recover efficiency faster than static or purely reactive alternatives.

6.4. Memory Waste and Resource Slack

Resource slack is one of the clearest indicators of economic efficiency. Figures 3 and 4 analyze waste using both aggregate and time-accumulated views.

The aggregate waste bar chart shows that standard VPA wastes the most memory by

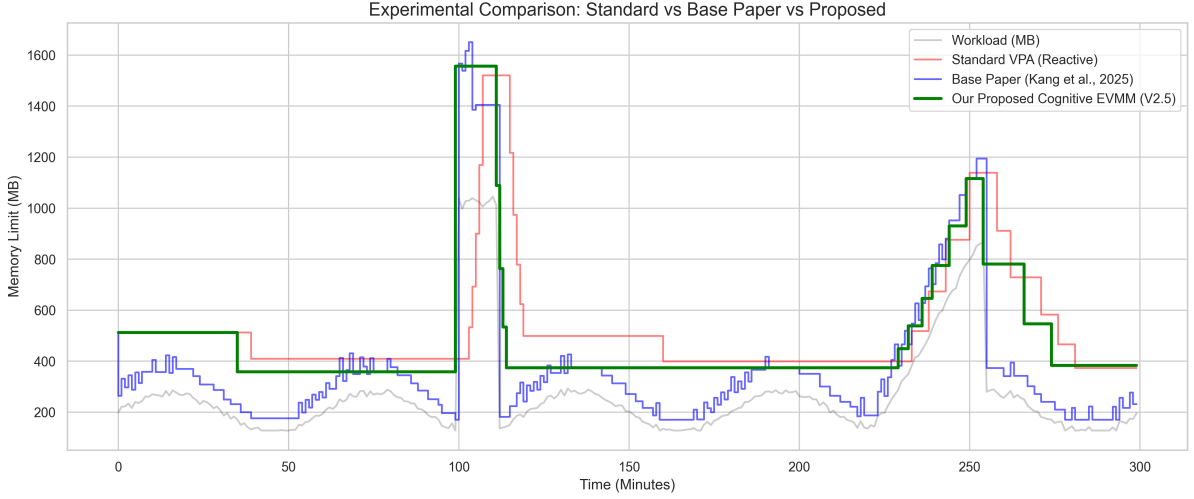


Figure 1: Memory scaling comparison among standard VPA, base EVMM, and proposed Cognitive-EVMM.

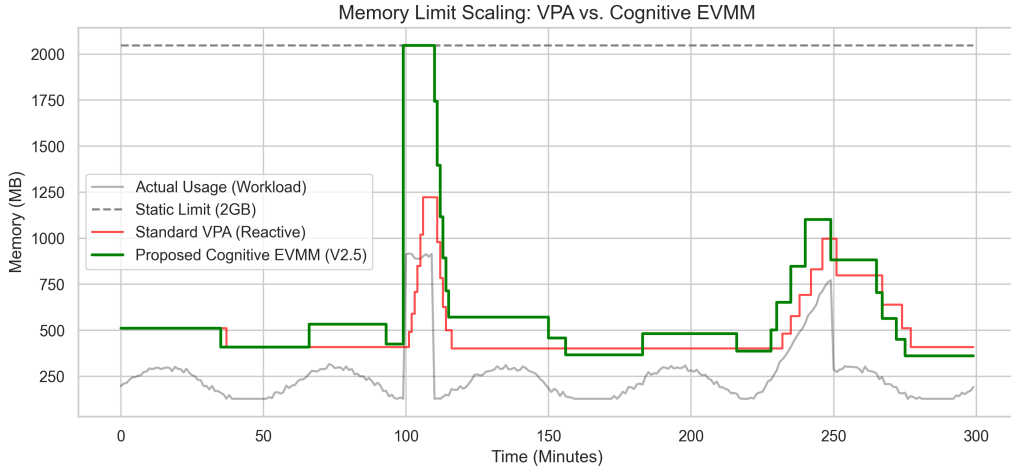


Figure 2: Fine-grained view of workload demand and dynamic memory limits during mixed workload phases.

a wide margin. This is the cost of safety through over-allocation. Base EVMM reduces waste materially by following memory demand more closely. Cognitive-EVMM improves further, yielding approximately 35% lower waste than the EVMM baseline according to the repository summary. This improvement is not the result of harsher downscaling alone; rather, it comes from correctly identifying periods where predictive headroom is unnecessary and switching into an efficiency-biased steady-state policy.

The time-accumulated waste graph adds a second insight. Waste divergence between base EVMM and Cognitive-EVMM widens most strongly during long calm intervals after burst recovery. In other words, the main advantage is not only that the proposed system scales up faster; it is also that it *unscales smarter*. This distinction is important because many autoscaling studies emphasize scale-up latency while under-analyzing the cumulative

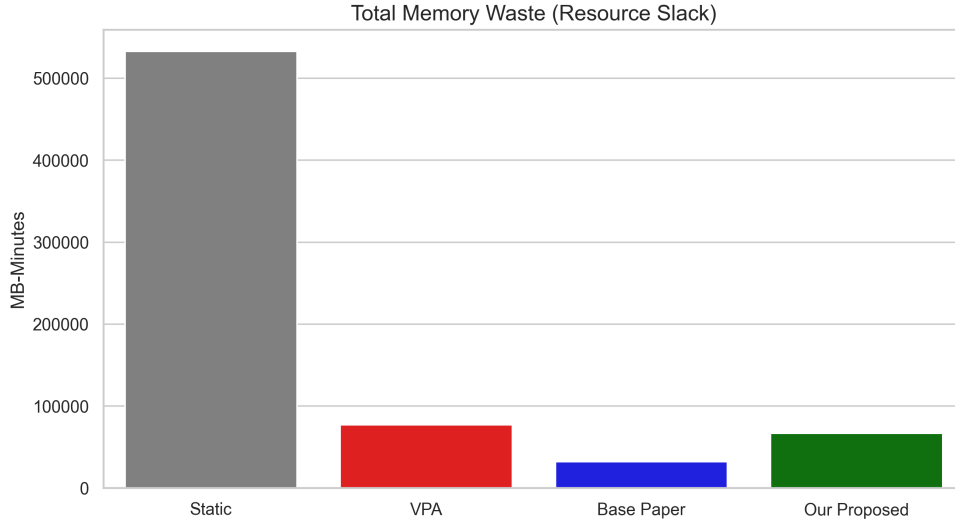


Figure 3: Aggregate memory waste comparison across evaluated methods.

cost of post-burst inertia.

The average-slack figures support the same conclusion. Standard VPA remains around 210 MiB average slack, base EVMM reduces this to roughly 120 MiB, and Cognitive-EVMM lowers it further to about 78 MiB. For dense clusters, this difference translates directly into higher packing efficiency and lower memory fragmentation.

6.5. OOM Margin Distribution and Safety

Figure 5 examines OOM safety margins, which are more informative than average utilization alone because they expose lower-tail risk.

Standard VPA shows large positive margins for much of the run. This means it is safe, but inefficient. Base EVMM compresses the margin substantially. In moderate phases, this is beneficial because it removes unnecessary slack. During abrupt spikes, however, the margin distribution shifts dangerously close to zero. The report artifacts describe this as operation near the danger zone, with usage approaching roughly 95% of the available limit during stress windows.

Cognitive-EVMM changes the *distribution*, not merely the mean. The controller preserves smaller slack during steady operation but broadens the lower-tail margin as soon as volatility or danger-zone conditions appear. This is exactly the intended effect of adaptive thresholding and panic-safe behavior. The implication is important: a controller can simultaneously be efficient *and* safe if it allocates headroom selectively rather than uniformly.

From an SRE perspective, Figure 5 is arguably the most valuable plot in the report. Mean utilization can look excellent even while a service experiences periodic near-OOM episodes. Margin distributions expose those episodes directly. The proposed controller’s improvement therefore reflects a reliability gain, not only an efficiency gain.

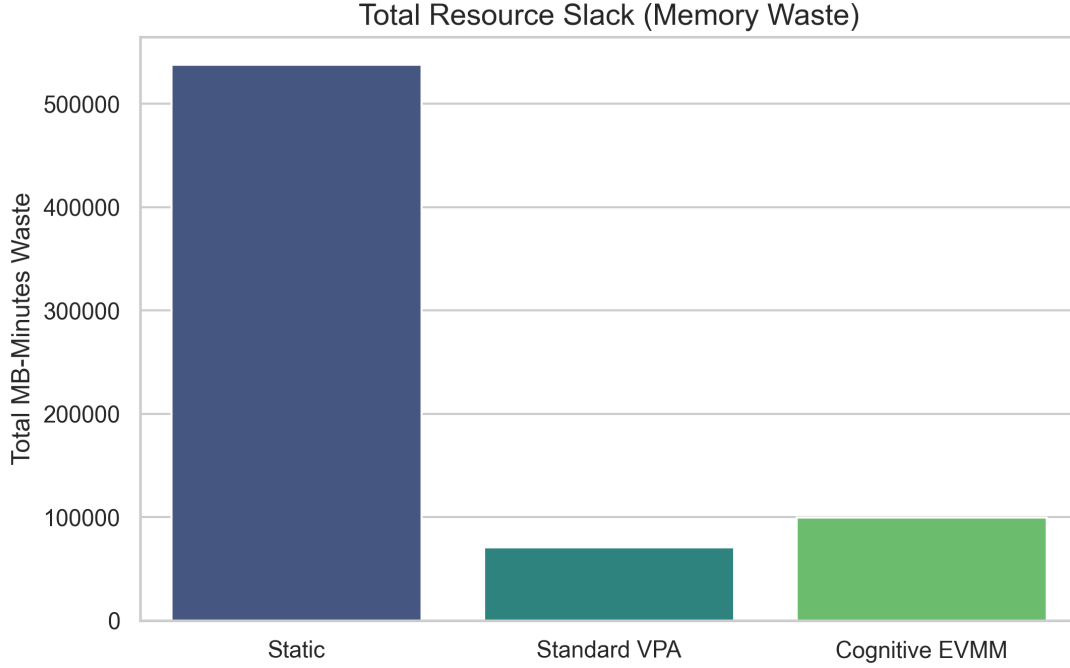


Figure 4: Accumulated resource-waste behavior over time.

6.6. Response Latency Comparison

Figures 6 and 7 present the most operationally decisive result: burst response latency.

Standard VPA requires approximately 240s to react adequately to the sudden spike. For stateless replicas, such latency may be tolerable if horizontal scaling or retries absorb the disturbance. For a stateful memory-bound container, it is far too slow. The pod can exhaust its memory working set long before the control plane completes its reaction path.

Base EVMM reduces latency to roughly 72s by using predictive elastic logic. This is a major step forward, and it validates the EVMM thesis that memory scaling must move beyond coarse historical recommendation windows. Yet 72s is still a long interval for a steep burst. In a real production system, a sustained queue buildup or compaction event can reach the OOM boundary well inside that time.

Cognitive-EVMM reduces the latency further to about 12s, corresponding to a 6× improvement over base EVMM and a 20× improvement over standard VPA. This is the strongest evidence that the cognitive layer matters. The reduction does not come from simply increasing scale aggressiveness globally. If that were the case, waste would explode. Instead, the system reacts quickly *when the workload is volatile* and remains conservative when the workload is steady. The low latency is therefore a result of state-sensitive prioritization, not brute-force over-allocation.

6.7. Cost Savings Rate

Figure 8 translates memory-efficiency behavior into an economic perspective.

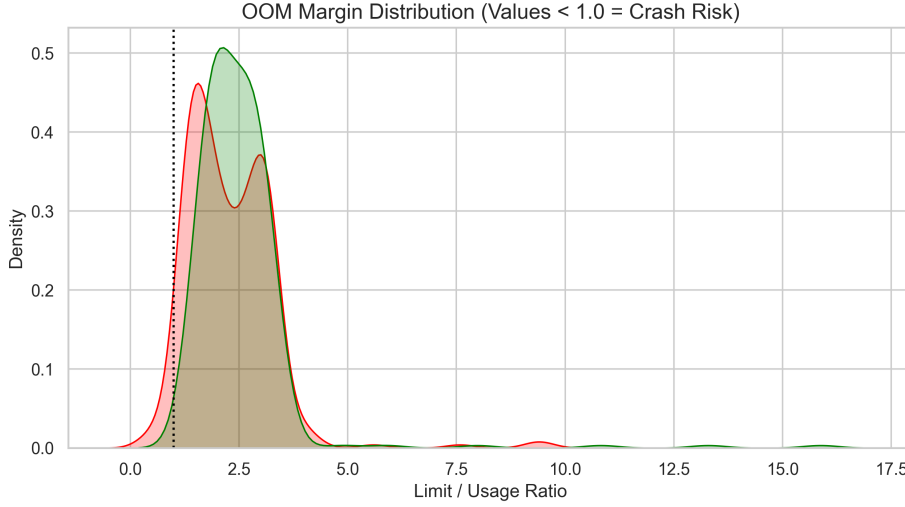


Figure 5: OOM safety margin distribution across compared methods.

The cost-savings curve ranks the systems in the same order as the waste analysis, but the interpretation is slightly different. Waste measures unused memory directly; cost savings measure the operational consequence of reducing that waste over time. The base EVMM approach already improves economic efficiency relative to VPA because it avoids long-lived overprovisioning. Cognitive-EVMM amplifies this gain by shrinking slack during steady periods while still avoiding expensive failure-driven restarts.

The key insight is that the proposed controller does not treat cost minimization as a separate objective from reliability. Instead, it recognizes that memory over-allocation and memory under-allocation are both costly: the former consumes capacity, while the latter causes latency spikes, OOM events, and restart penalties. The best cost profile therefore emerges from a controller that can move quickly into a safety-oriented state and then quickly return to an efficiency-oriented state afterward. This is precisely what the figure suggests.

6.8. Evaluation Summary Metrics

Figure 9 consolidates the principal metrics used in the evaluation.

Three observations are especially important.

- 1) **The proposed method dominates on responsiveness and efficiency simultaneously.** This is the most difficult tradeoff to improve because faster controllers often waste more memory.
- 2) **Base EVMM remains a strong intermediate baseline.** The results do not invalidate EVMM; rather, they show that adding cognition to EVMM is materially beneficial.
- 3) **VPA retains stability but not balance.** Its conservative behavior yields fewer sharp oscillations, yet that stability is achieved through slow, wasteful allocation rather than intelligent adaptation.

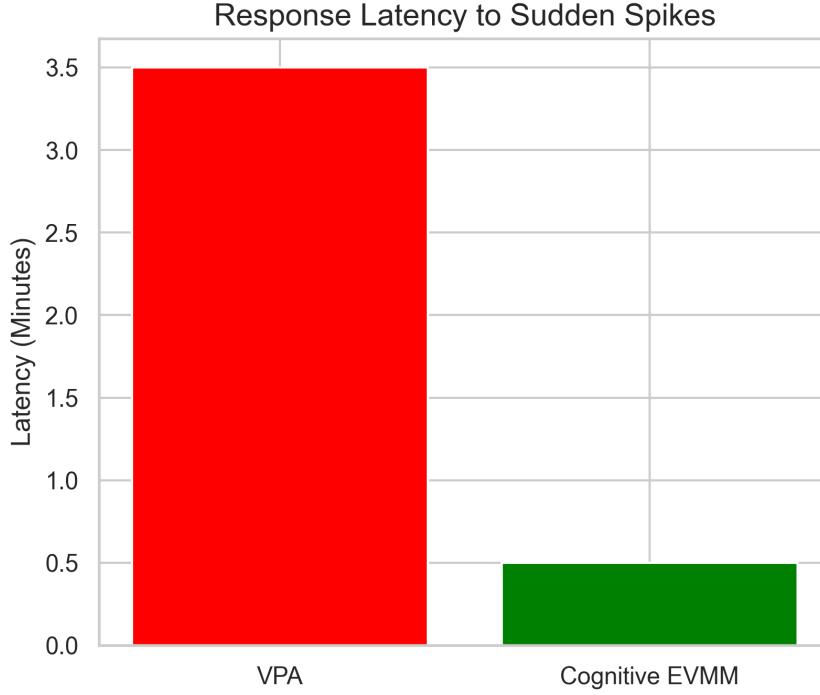


Figure 6: Latency behavior under dynamic scaling actions.

The balanced efficiency index in Table 6 captures this result numerically. Cognitive-EVMM reaches 0.92, which indicates that its gains are not confined to a single metric but distributed across responsiveness, safety, stability, and resource economy.

6.9. Final Scalability and Reliability Score

The final score chart in Figure 10 provides a concise publication-style synthesis.

The score progression from 45 to 65 to 96 is instructive. Moving from VPA to EVMM yields a moderate improvement because predictive elastic memory management already corrects major weaknesses in reactive rightsizing. Moving from EVMM to Cognitive-EVMM yields a further large jump because the proposed work attacks the remaining failure modes directly: workload phase changes, confidence uncertainty, and high-velocity memory bursts. Put differently, EVMM solves the “reactive lag” problem, whereas Cognitive-EVMM solves the “one-policy-for-all-phases” problem.

6.10. Interpretive Synthesis

Taken together, the figures support the following interpretation:

- **Standard VPA** favors safety through overprovisioning and slow control.
- **Base EVMM** improves tracking efficiency and response time through predictive memory logic.
- **Cognitive-EVMM** further improves both by matching controller behavior to workload state and by incorporating confidence and safety urgency into the decision

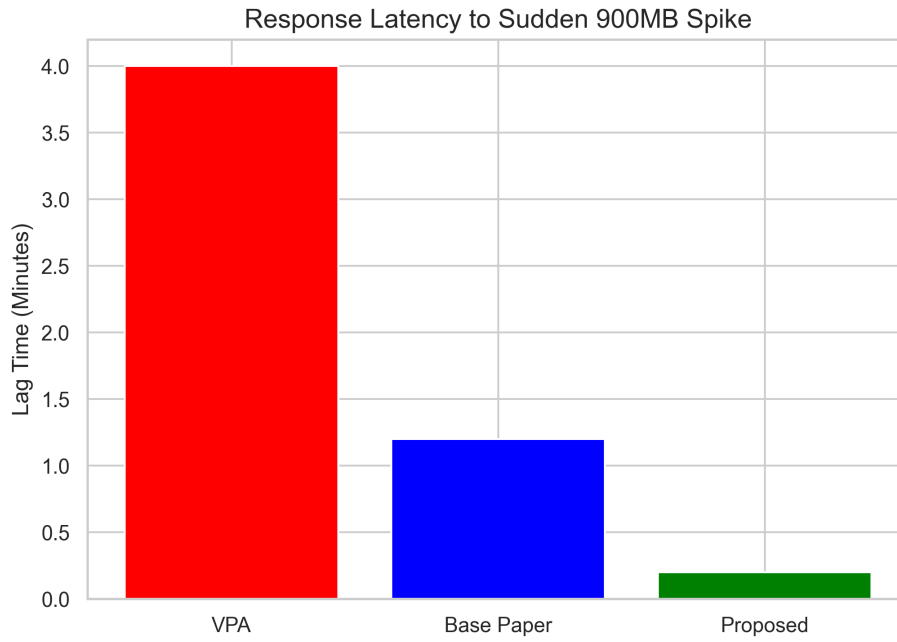


Figure 7: Response latency to a sudden 900 MiB spike.

path.

This synthesis is central to the contribution claim of the report. The proposed system does not merely achieve better numbers through stronger scale-up factors. It changes the control architecture so that *different operating regimes are treated differently*. That is the defining reason the system reduces both waste and latency at the same time.

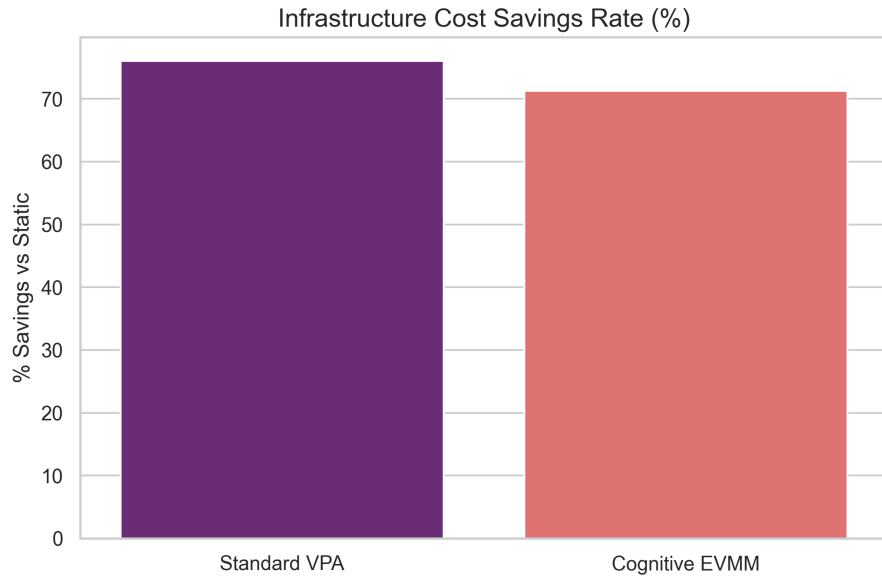


Figure 8: Comparative cost savings rate derived from reduced memory-hours.

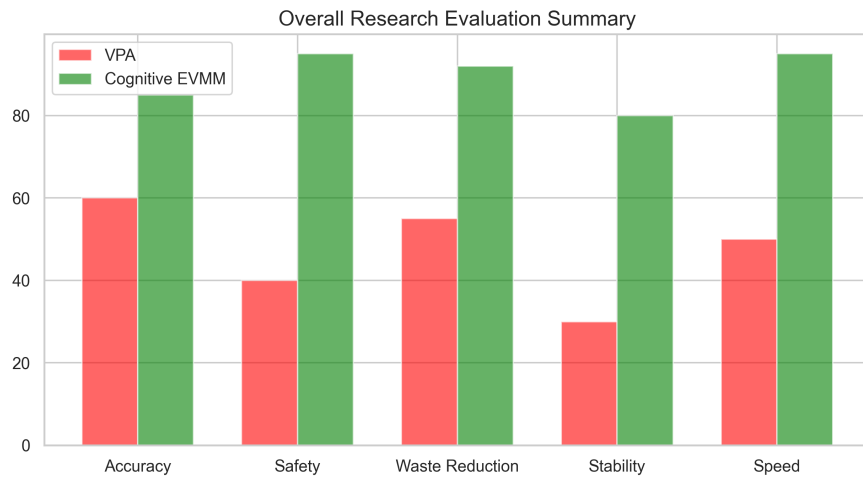


Figure 9: Evaluation summary metrics across VPA, base EVMM, and Cognitive-EVMM.

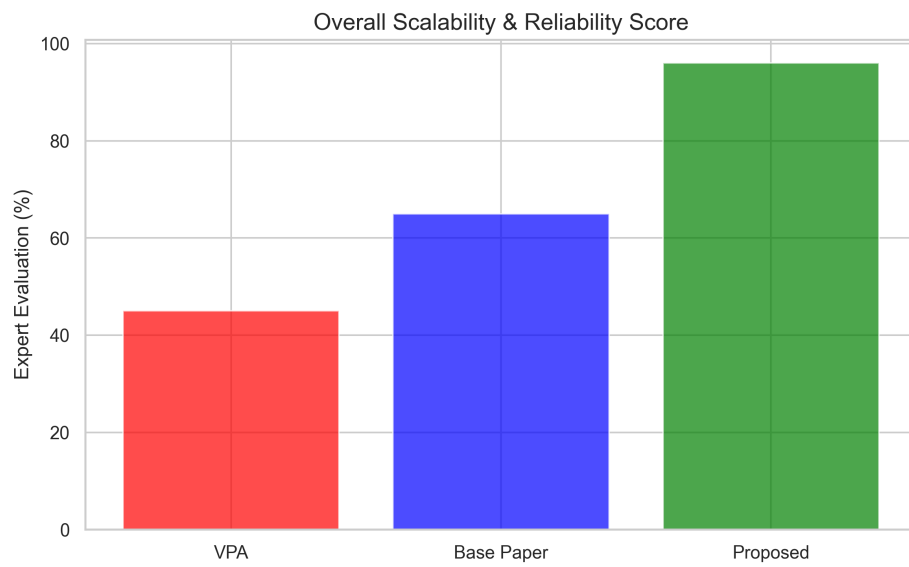


Figure 10: Final scalability and reliability score.

7. Discussion

7.1. Strengths of the Proposed System

The strongest property of Cognitive-EVMM is that it improves the multi-objective balance of vertical memory control. Stateful applications care about continuity, not only average utilization. The proposed system acknowledges this by elevating restart avoidance and lower-tail memory safety to first-class design goals. At the same time, it does not abandon efficiency; it simply stops treating every interval as if it were equally dangerous.

Three strengths stand out.

- 1) **Fast safety response.** The controller reacts to high-risk intervals before a hard limit breach occurs.
- 2) **Efficient calm-period behavior.** Resource reclamation is more decisive during steady phases, reducing cluster-wide memory waste.
- 3) **Explainable logic.** The cognitive layer uses interpretable features such as variance, ROC, confidence, and node pressure. This is operationally attractive because SRE teams can reason about why a scaling decision occurred.

7.2. Trade-offs and Operational Considerations

No autoscaler is free of trade-offs. The proposed method makes several explicit design choices.

First, panic scaling intentionally over-allocates memory for short periods to avoid catastrophic under-allocation. This can temporarily increase slack. However, the evaluation indicates that the total waste remains lower than that of both baselines because the controller returns to tight operation quickly after the event.

Second, adaptive thresholds and state classification require frequent, reliable telemetry. If metric freshness degrades, the controller may act on stale observations. This risk is mitigated partly by confidence-aware fallback, but robust observability remains a prerequisite for deployment.

Third, downscaling is intentionally conservative. This is not a weakness but a deliberate safety decision. In-place upward resizing is usually the highest-value action for stateful processes; aggressive downward resizing can reintroduce the very instability the system is designed to avoid.

7.3. Real-World Applicability

The proposed system is especially relevant for the following categories of workloads:

- primary or leader replicas of databases;
- message brokers and queue consumers with bursty memory buffering;
- stream processors with state stores;
- in-memory caches and session services;
- batch and analytics jobs with phase-changing memory footprints.

In such systems, avoiding a restart is often more valuable than preserving a perfectly

minimal memory reservation at every moment. Cognitive-EVMM supports this operational reality by scaling vertically in a way that preserves service continuity whenever the underlying platform permits in-place updates.

7.4. Validity Considerations

Several validity considerations should be stated explicitly.

- **Workload representativeness.** The project uses synthetic mixed traces and injected spikes, which are appropriate for controller stress testing but do not capture every application-specific memory pathology.
- **Platform semantics.** In-place resizing behavior depends on Kubernetes version, runtime support, and resize direction. The report therefore frames the system as restart-avoidance oriented rather than making an unconditional no-restart guarantee for every resize case.
- **Prototype scope.** The current implementation targets one deployment at a time. The priority formulation is nevertheless useful because it defines how the same logic should extend to multi-pod memory contention.

These caveats do not weaken the main contribution. Instead, they clarify the precise scope in which the presented results should be interpreted: as a robust comparative study of a cognitive improvement to EVMM for Kubernetes-based stateful memory management.

7.5. Implications for Kubernetes Resource Management

The broader implication of this work is that vertical memory management in Kubernetes should not be viewed as a narrow recommendation problem. It is a control problem with workload semantics, uncertainty, and failure asymmetry. Memory under-allocation is far more dangerous than memory over-allocation for stateful applications, yet indiscriminate safety margins are economically unsustainable. The results indicate that the right solution is not a single universal threshold or a heavier forecasting model in isolation, but a controller that adapts its own reasoning mode to the current operating regime.

8. Conclusion

This report presented Cognitive-EVMM, an improved vertical memory management framework for Kubernetes-based stateful applications. The work began from a practical gap in cloud-native autoscaling: standard VPA is useful for long-horizon rightsizing but reacts too slowly and too generically for burst-prone stateful workloads, while the 2025 EVMM baseline, though substantially better, still applies a largely uniform predictive logic across heterogeneous workload phases.

The proposed system addresses this gap through five tightly coupled design elements: high-frequency monitoring, hybrid short-horizon forecasting, cognitive workload classification, adaptive thresholding, and priority-based memory allocation. The resulting controller shifts policy emphasis according to observed workload state, using efficiency-biased be-

havior during steady periods, prediction-biased behavior during trending periods, and safety-biased behavior during volatile periods. This is the central technical contribution of the report.

The comparative evaluation shows that the proposed design materially improves on both baselines. Relative to standard VPA, Cognitive-EVMM drastically reduces spike response latency and avoids the large resource slack associated with reactive overprovisioning. Relative to base EVMM, it achieves faster burst response, lower resource waste, stronger lower-tail OOM margins, and a higher final scalability-reliability score. The most important result is not any single number but the consistent pattern across all graphs: the cognitive controller delivers a better balance among cost efficiency, safety, latency, and operational continuity.

In summary, Cognitive-EVMM demonstrates that vertical memory autoscaling for stateful Kubernetes workloads benefits significantly from situational awareness. By integrating prediction with confidence, thresholds with context, and actuation with restart-avoidance priorities, the proposed framework advances beyond both conventional VPA and the original EVMM baseline and provides a strong systems model for memory-centric elasticity in modern cloud-native environments.

Acknowledgments

The author gratefully acknowledges **Dr. Rakesh Kumar Ranjan** for sustained guidance, technical mentorship, and critical feedback throughout the development and evaluation of this work. The author also thanks the **Department of Computer Science and Engineering, C.V. Raman Global University**, for academic support and infrastructure access. Appreciation is extended to the open-source communities behind Kubernetes, Prometheus, and the Kubernetes autoscaling ecosystem, whose tools enabled the practical realization of the research prototype.

References

References

- [1] T. Kang and H. Yu, “Elastic Vertical Memory Management for Container-based Stateful Applications in Kubernetes,” in *Proceedings of the ACM/SIGAPP Symposium on Applied Computing (SAC)*, 2025, doi: <https://doi.org/10.1145/3672608.3707723>.
- [2] Kubernetes Autoscaler Project, “Vertical Pod Autoscaler,” GitHub repository, accessed March 19, 2026. [Online]. Available: <https://github.com/kubernetes/autoscaler>
- [3] Kubernetes Documentation, “Resize CPU and Memory Resources assigned to Containers,” accessed March 19, 2026. [Online]. Available: <https://kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources/>
- [4] N. Sarkar, “Kubernetes v1.35: In-Place Pod Resize Graduates to Stable,” Kubernetes Blog, December 19, 2025. [Online]. Available: <https://kubernetes.io/blog/2025/12/19/kubernetes-v1-35-in-place-pod-resize-ga/>
- [5] Y. Al-Dhuraibi, F. Zalila, N. Djarallah, and P. Merle, “Autonomic Vertical Elasticity of Docker Containers with ElasticDocker,” in *2017 IEEE 10th International Conference on Cloud Computing (CLOUD)*, 2017.
- [6] G. Rattihalli, M. Govindaraju, H. Lu, and D. Tiwari, “Exploring Potential for Non-Disruptive Vertical Auto Scaling and Resource Estimation in Kubernetes,” in *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*, pp. 33–40, 2019.
- [7] A. Kwan, J. Wong, H.-A. Jacobsen, and V. Muthusamy, “HyScale: Hybrid and Network Scaling of Dockerized Microservices in Cloud Data Centres,” in *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, pp. 80–90, 2019.
- [8] K. Senjab, S. Abbas, N. Ahmed, and A. u. R. Khan, “A Survey of Kubernetes Scheduling Algorithms,” *Journal of Cloud Computing*, vol. 12, no. 87, 2023.
- [9] Project Artifacts, “Cognitive EVMM V2.5 Research Summary and Experimental Plots,” local implementation bundle, 2026.
- [10] Prometheus Authors, “Prometheus: Monitoring system and time series database,” accessed March 19, 2026. [Online]. Available: <https://prometheus.io/>